Universidade Federal de Goiás
Instituto de Informática - INF

Implantação e Entrega de Software – 2026/2
Curso: Engenharia de Software
Professora: Sofia Costa Paiva

Atividade prática: Refatoração

Você e sua equipe assumiram a sustentação do núcleo financeiro da plataforma **SaaS Billing Co.** O sistema realiza o cálculo mensal de assinaturas, aplicação de cupons promocionais, cobrança via cartão de crédito e envio de recibos por e-mail.

O código atual foi desenvolvido sob forte pressão de prazo por equipes anteriores, resultando em um acúmulo severo de débito técnico de código e de arquitetura. A diretoria aprovou uma janela de manutenção para pagar essa dívida técnica, sob uma restrição estrita: nenhum comportamento de negócio existente pode ser quebrado e a esteira de integração contínua (CI) deve executar os testes unitários sem depender de infraestrutura externa (bancos de dados locais ou servidores SMTP).

Com base no código anexo da plataforma Saas Billing Co., faça, em duplas:

1. Matriz de Code Smells e Violações Arquiteturais: Preencha uma tabela contendo pelo menos 5 problemas estruturais distintos, detalhando:

- Linha/trecho do código afetado.
- Nome do *Code Smell* ou princípio violado (ex.: *Long Method*, *Primitive Obsession*, violação de *Single Responsibility Principle*, dependência direta de I/O, etc.).
- Os "juros" decorrentes: qual o impacto prático dessa decisão na manutenibilidade e no ciclo de entrega contínua?

2. Refatoração: Aplique operações formais do catálogo de refatorações (utilizando, preferencialmente, atalhos automatizados da sua IDE) para desacoplar a solução:

1. **Eliminação de *Primitive Obsession* e Imutabilidade:**
    - Crie o *Value Object* imutável `Money` (usando `record` ou classe imutável) para encapsular arredondamentos e operações financeiras.
    - Elimine parâmetros soltos encapsulando a entrada em um objeto de comando/parâmetros (`BillingCommand`).
2. **Inversão de Dependência e Desacoplamento de I/O (Arquitetura Hexagonal / Portas):**
    - Extraia interfaces para os serviços externos: `CustomerRepository`, `PaymentGateway` e `NotificationService`.
    - Injete uma instância de `java.time.Clock` para eliminar o uso de `LocalDate.now()` estático, tornando os cálculos sensíveis a datas 100% determinísticos.

3. **Extração de Classes e Métodos:**
    - Isole as regras matemáticas de precificação em uma classe de domínio pura (`PricingCalculator`), livre de efeitos colaterais de rede ou banco.
    - Reduza a complexidade ciclomática do método principal, garantindo que o método orquestrador atue como uma fachada de alto nível com no máximo 25 linhas legíveis.
4. **Remoção de Código Morto e Limpeza de Erros Mascarados:**
    - Elimine blocos de captura genéricos (`catch (Exception e)`) que silenciam falhas de infraestrutura sem tratamento adequado.

3. Histórico Estruturado no Git: O repositório deve conter um histórico de *commits* rastreável demonstrando a evolução passo a passo (evite *commits* únicos do tipo "código final"):

- `commit 1`: Adição do código legado base.
- `commit 2`: Introdução dos Value Objects e remoção de Primitive Obsession.
- `commit 3`: Extração das interfaces de infraestrutura e injeção de dependência.
- `commit 4`: Refatoração do motor de precificação e validação da suíte verde.

Cada dupla deverá submeter no SIGAA:

1. **Link do Repositório no GitHub** (público ou com acesso para professora).
2. **Relatório Técnico (`RELATORIO.md`)** – pode estar no mesmo repositório no GitHub - contendo:
    - Tabela de diagnóstico do débito técnico (item 1).
    - Tabela comparativa de métricas antes vs. depois da refatoração (item 2).